

C++&Qt Day3 hw2 AI 사용 보고서

휴머노이드 수습단원 이주원

목차

1. ai사용 부분
2. 대화내용
3. 깨달은 점

1. ai 사용 부분

이번 과제에서 ai가 사용된 부분은 크게 6가지이다.

첫 번째는 알고리즘을 Qt로 가져오는 부분이다. A*와 다익스트라 자체는 이미 알고 있는 내용이었지만 이걸 콘솔이 아니라 위젯 위에 올리려니 어디서부터 손을 대야 할지 감이 안 잡혔다. 그래서 코드를 그대로 받아 쓰지는 않고 구조만 참고하였다. 특히 맵을 2차원 배열이 아니라 1차원 vector로 잡고 $i = y * w + x$ 로 인덱스를 계산하는 방식을 참고해서 내 코드에 맞게 다시 썼다.

두 번째는 커스텀 위젯을 만드는 부분이다. QWidget을 상속받아 직접 그리는 위젯을 만들어야 하는데, paintEvent를 override 한다는 것까지는 알았지만 이걸 내가 직접 부르면 되는 줄 알고 호출했다가 화면이 안 바뀌어서 질문하였다.

세 번째는 Qt Designer에서 만든 위젯을 map_widget으로 바꾸는 부분이다. 화면에 위젯을 올려두었는데 ui->map_area->run_astar() 가 컴파일이 안 되어서 물어보았다.

네 번째는 twin 부분이다. 두 알고리즘을 비교하려면 맵 두 개가 완전히 똑같아야 하는데, 한쪽에 벽을 찍었을 때 반대쪽에도 같이 찍히게 하는 방법을 몰랐다. 처음에는 MainWindow에서 두 위젯을 다 관리하는 방식을 생각했는데 그러면 코드가 지저분해질 것 같아서 물어보았다.

다섯 번째는 우선순위 큐 선언 부분이다. 비용이 작은 것부터 꺼내야 하는데 priority_queue는 기본이 큰 것부터 나오기 때문에 이걸 어떻게 바꾸는지 몰라서 질문하였다.

```
std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>, std::greater<>>> open;
```

이 선언이 길어서 그냥 외워 쓰기보다 각 자리가 무슨 뜻인지 같이 물어보았다.

여섯 번째는 시간을 재는 부분이다. 두 알고리즘의 성능을 비교해서 보여주고 싶은데 Qt에서 시간을 어떻게 재는지 몰라서 질문하였다. 그리고 실제로 재봤더니 방문 노드는 A*가 적는데 시간은 오히려 더 길게 나와서 이게 왜 그런지 물어보았다.

2. 대화내용

Qt 경로탐색 시각화 위젯 구현 ▾

Qt로 A*랑 다익스트라 경로탐색 시각화 만들려고 하는데, QWidget 상속받은 커스텀 위젯에 격자맵 그리고 마우스로 벽 찍는 구조를 어떻게 잡아야 돼? 맵 데이터는 2차원 배열로 들고 있으면 되나?

메모리 불러옴 >

2차원 배열로 들고 있는 건 맞는데, "맵 데이터"와 "알고리즘 상태"를 같은 배열에 섞지 않는 게 제일 중요하니다. 벽 배열에 방문표시까지 같이 넣으면 재실행할 때마다 벽이 날아가거나 지우는 코드가 지저분해져요.

구조 분리

- `grid_map` — 순수 데이터 (폭/높이, 벽 배열, 시작/도착). Qt 의존성 없음
- `map_widget : QWidget` — 그리기 + 마우스 입력만
- 알고리즘 — `grid_map`을 받아서 결과 구조체 반환하는 독립 함수/클래스

이렇게 해두면 알고리즘을 콘솔로도 테스트할 수 있고, A*랑 다익스트라 갈아끼우기도 쉽습니다.

2차원 배열보다 **1차원으로 펴서 `y * w + x`로 접근**하는 걸 추천해요. A*에서 `came_from[]`, `g_cost[]` 같은 부속 배열을 인덱스 하나로 다루게 되고, 우선순위 큐에 int 하나만 넣으면 됩니다.

```
cpp
```

다운로드

+

 메시지를 입력하세요...

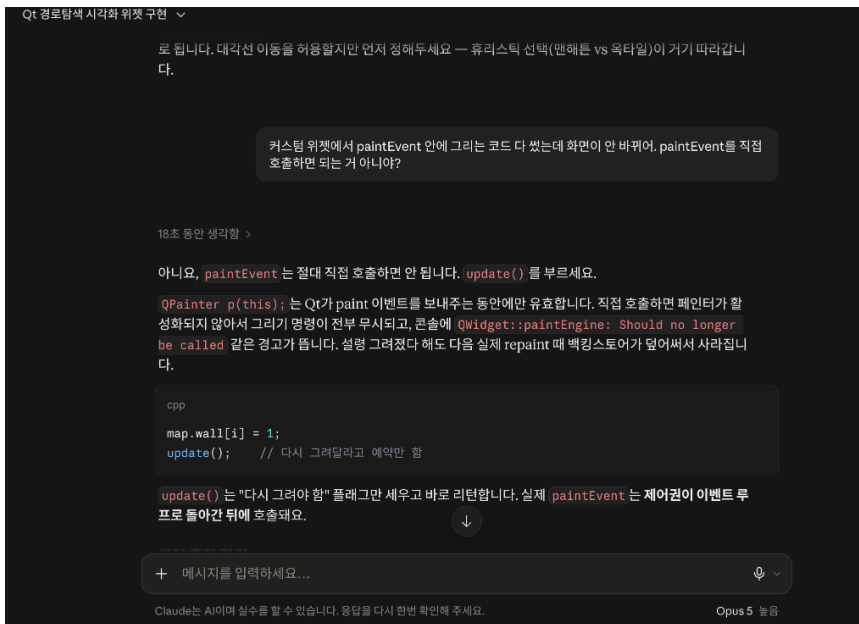
🔊

🗣️

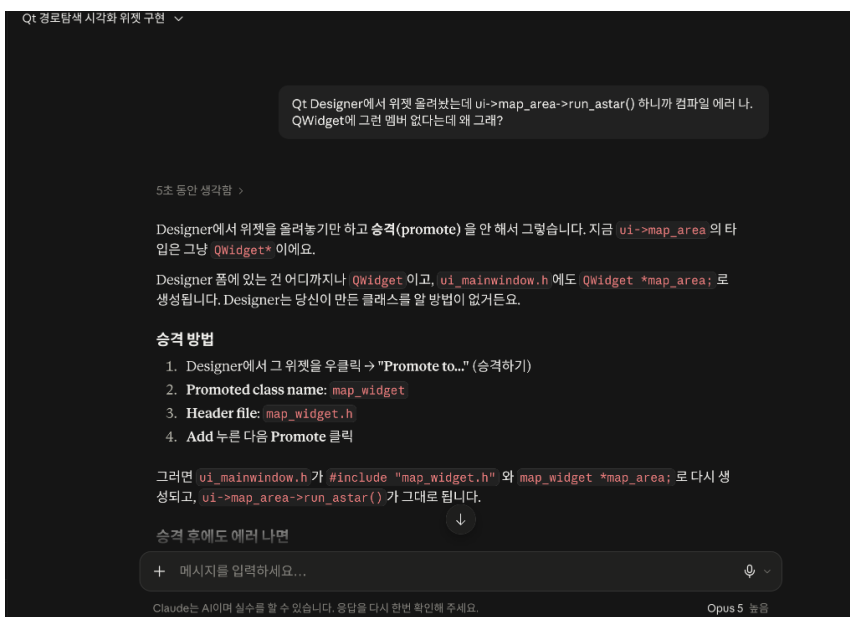
▾

Claude는 AI이며 실수를 할 수 있습니다. 응답을 다시 한번 확인해 주세요.

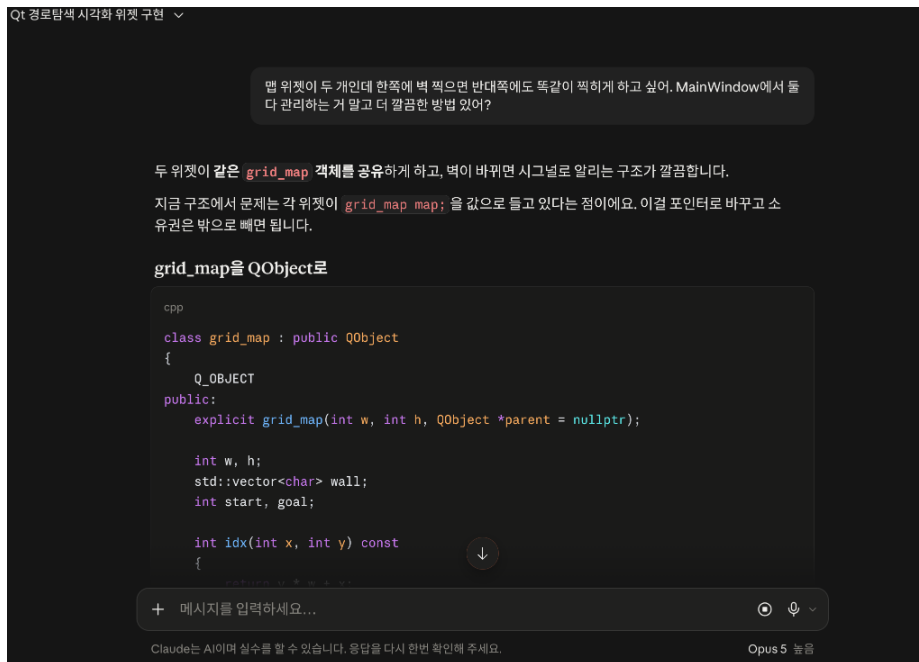
Opus 5 높음



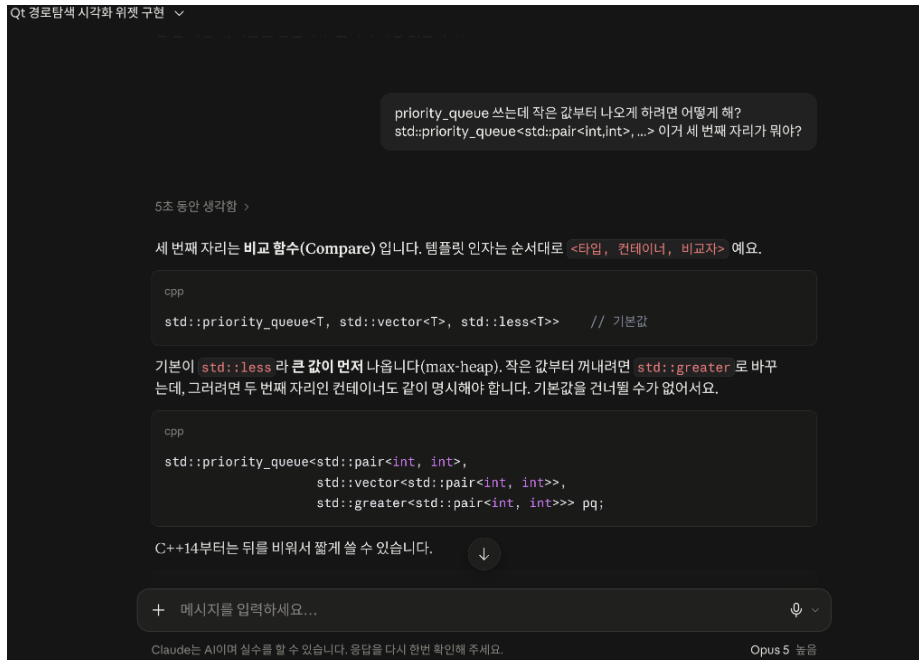
paintEvent를 직접 부르면 되는지 물어본 부분이다. 그게 아니라 update()를 부르면 Qt가 알
아서 paintEvent를 불러 준다고 하였다.



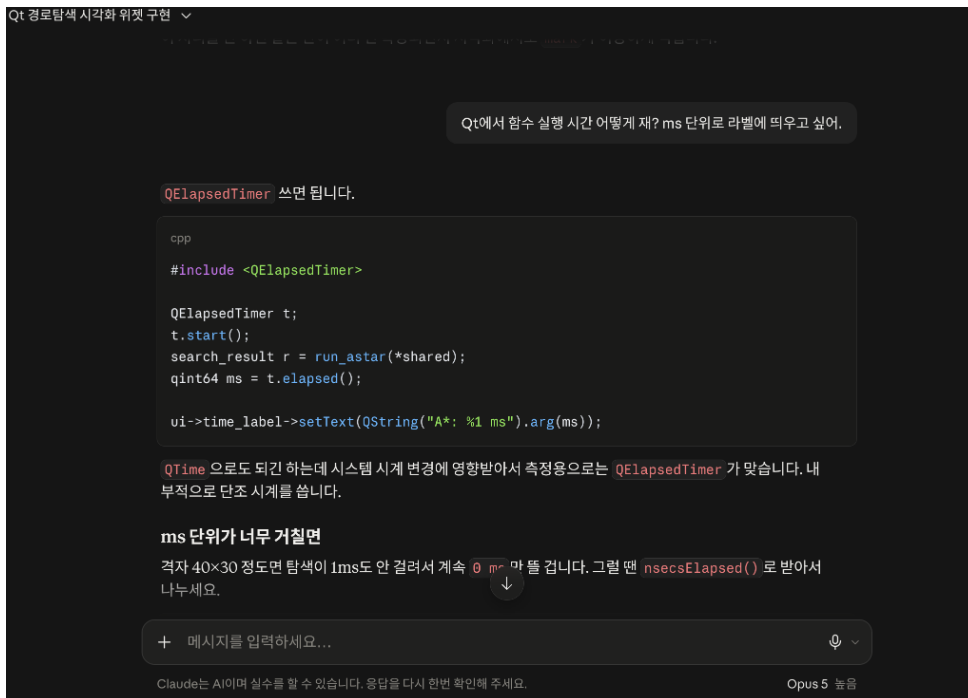
디자이너로 올린 위젯에서 내가 만든 함수를 못 쓰는 문제를 물어본 부분이다. 승격
(promote)을 해줘야 그 위젯이 QWidget이 아닌 map_widget으로 취급된다는 답을 받았다.



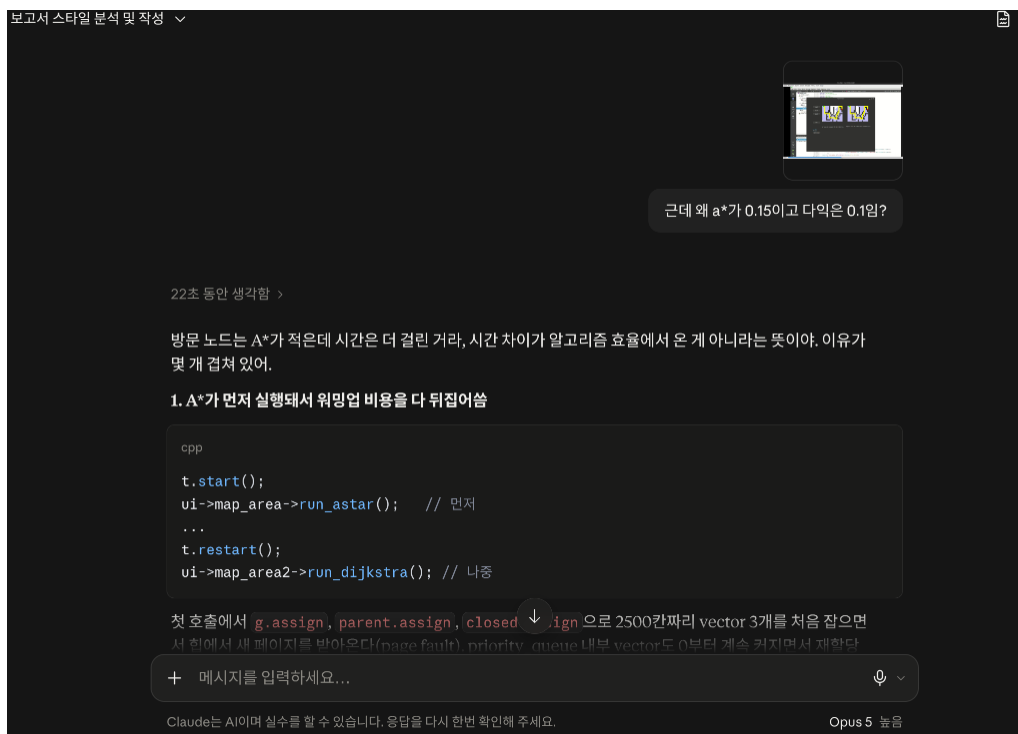
처음에는 두 위젯이 하나의 맵 객체를 포인터로 공유하고 벽이 바뀔 때 시그널을 쏘는 방식을 제안받았다. 하지만 위젯이 두 개뿐이라 굳이 클래스를 하나 더 만들 필요는 없다고 판단하여, 위젯이 서로를 가리키는 twin 포인터를 하나씩 들고 상대방 함수를 직접 부르는 방식으로 단순화해서 구현하였다. 대신 위젯이 세 개 이상으로 늘어나면 이 방식은 연결이 복잡해져서 공유 객체 방식이 유리하다는 것도 함께 알게 되었다.



`priority_queue` 선언에서 세 번째 자리에 `std::greater<>` 를 넣으면 작은 값부터 나온다는 것을 설명받은 부분이다.



시간을 재는 방법을 물어본 부분이다. QElapsedTimer를 쓰고 nsecsElapsed()로 나노초를 받아 나눠 쓰면 된다고 하였다.



A*가 방문 노드는 적은데 시간이 더 걸리는 이유를 물어본 것이다.

3. 깨달은 점

- `paintEvent`는 내가 부르는 함수가 아니라 Qt가 필요할 때 불러 주는 함수라는 것을 깨달았다. 그래서 화면을 바꾸고 싶으면 `update()`로 다시 그려 달라고 요청만 하고, 실제로 그리는 시점은 Qt가 정한다. Day2의 `refresh_view`는 내가 직접 부르는 함수였는데 이번엔 반대로 해서 처음엔 헷갈렸다. 그리는 순서가 곧 겹치는 순서라는 것도 알게 되었는데, 벽과 길을 먼저 칠하고 그 위에 방문한 칸, 경로, 시작점, 도착점 순으로 덧칠해야 원하는 대로 보인다.

- 디자이너로 올린 위젯은 승격을 해줘야 내가 만든 클래스가 된다는 것을 알게 되었다. 승격을 하면 `.ui` 파일 아래쪽에 `customwidget` 항목이 생기면서 헤더를 참조하게 되는데, 이렇게 눈에 잘 안 보이는 곳에서 연결이 이루어진다는 점이 처음엔 낯설었다.

- 위젯끼리 서로를 가리키는 포인터를 들고 있으면 한쪽에서 다른 쪽 함수를 바로 부를 수 있다는 것을 알게 되었다. `ui->map_area->twin = ui->map_area2;` 이렇게 생성자에서 한 번만 연결해 주면 그 뒤로는 `mousePressEvent` 안에서 `twin->set_wall(i);` 처럼 짧게 쓸 수 있다. `MainWindow`가 전부 관리하는 것보다 훨씬 깔끔했다.

- `priority_queue`의 세 번째 템플릿 인자가 비교 방식을 정한다는 것을 알게 되었다. 기본이 큰 값부터 나오는 이유도 여기에 있었고, `std::greater<>` 를 넣으면 작은 값부터 나온다. 그리고 `pair`를 넣으면 `first`를 기준으로 정렬되기 때문에 비용을 `first`에, 칸 번호를 `second`에 넣는 이유도 이해할 수 있었다.

- 두 알고리즘을 비교할 때는 `ms`보다 `visited`가 더 적합한 지표라는 것을 알게 되었다. `visited`는 몇 번을 돌려도 같은 값이 나오지만 `ms`는 캐시 상태나 실행 순서에 따라 계속 흔들리기 때문이다. 실제로 A를 먼저 돌리는 구조라 A가 메모리 할당 비용을 다 떠안아 손해를 보고 있었다. 실험으로 복잡도를 확인하려면 한 번 재서 비교하는 게 아니라 맵 크기를 바꿔가며 늘어나는 추세를 봐야 한다는 것도 배웠다. 이는 이미 전 보고서에서 배운 내용이렸으나 실전에서 적용할 때 한 번 더 알려주어서 좋았다.